

# **Multiplicador 4×4 sem sinal, com detecção de overflow**

Descrição estrutural em Alliance CAD (genlib), verificação exaustiva com asimut, layout físico (ocp/nero/graal) e atraso de propagação medido no nível lógico e por conferência SPICE (spiceopus).

## **1. Objetivo**

Projetar um multiplicador combinacional de dois operandos de 4 bits sem sinal (A, B entre 0 e 15), produzindo o produto de 8 bits  $P = A \times B$  e um sinal OVF que indica overflow ( $P > 15$ ). O circuito foi descrito estruturalmente, porta por porta, usando a linguagem genlib do Alliance CAD, e verificado contra as 256 combinações possíveis de entrada.

## **2. Arquitetura**

A implementação segue o modelo de array multiplier: uma matriz de produtos parciais gerada por 16 portas AND2 (a2\_x2), somada por diagonais com meios-somadores e somadores completos (xr2\_x1, o2\_x2) até produzir os oito bits do produto. A primeira diagonal é somada por meios-somadores; as diagonais seguintes, por somadores completos, propagando o carry até a última linha, que fecha P4 a P7. Os oito bits de saída passam por buffers (buf\_x2) até os conectores externos. OVF é o OR dos quatro bits mais altos do produto (P4 a P7).

## **3. Ambiente e ferramentas**

O fluxo roda em Docker (Ubuntu 18.04 + Alliance CAD 5.1.1), com um script para cada etapa (build\_mult4.sh, run.sh, place\_route.sh, find\_delay.sh). O binário asimut da variante amd64 do pacote trava com segmentation fault em qualquer netlist estrutural com instanciação de componente; a variante i386 funciona e foi usada no lugar. O gerador de VHDL do genlib produz identificadores de barramento inválidos, corrigidos depois da geração. As ferramentas de layout físico (ocp, nero, graal) esperam o arquivo de tecnologia em um caminho diferente de onde o pacote instala. O SpiceOpus exige uma versão do Ubuntu mais nova que o restante do fluxo e roda por isso em um container separado.

## **4. Metodologia de verificação**

O resultado de cada um dos 256 casos simulados no asimut foi comparado com o valor esperado em mult4\_table.txt, uma tabela de referência simples (A, B, P, OVF) com uma linha por combinação.

## **5. Resultados**

Todos os 256 vetores de teste passaram — nenhuma divergência em P ou OVF contra a tabela de referência.

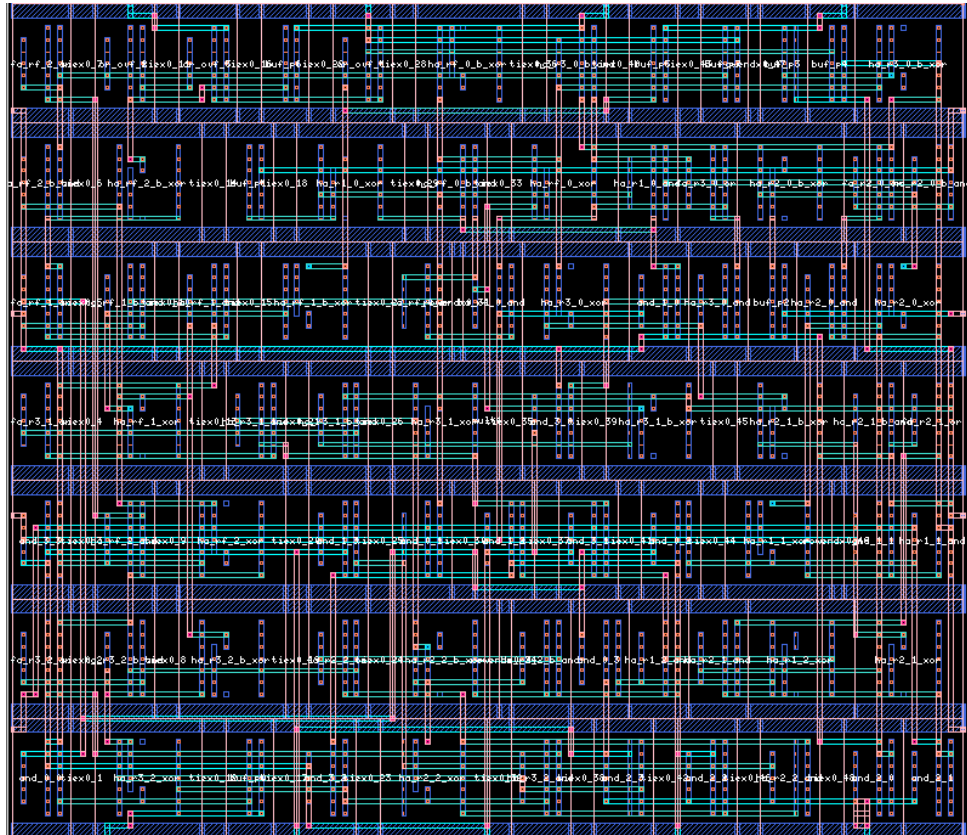


Figura 1 — Layout físico do mult4 após posicionamento (ocp) e roteamento (nero), capturado do graal: 127 células em 7 fileiras.

## 6. Layout físico e atraso de propagação

O netlist foi posicionado (ocp) e roteado (nero) em um layout físico real (Figura 1). O atraso de propagação foi medido com `find_delay.sh`: ele roda o conjunto exaustivo de 256 casos no asimut com tempos de acomodação decrescentes entre forçar as entradas e checar as saídas, até a simulação começar a errar.

| Acomodação | Erros |
|------------|-------|
| 13 ns      | 0     |
| 12 ns      | 1     |
| 11 ns      | 8     |
| 10 ns      | 25    |

O resultado é 13 ns, usando o atraso declarado nos modelos comportamentais da sxlíb (after 1000 ps em cada célula), para a transição de pior caso ( $0 \times 0 \rightarrow 15 \times 15$ ). Como conferência, o

netlist extraído do layout roteado (cougar -t -ac, 578 transistores, capacitâncias parasitas reais) foi simulado no spiceopus com um modelo de transistor genérico (a tecnologia do Alliance usada aqui só tem regras geométricas, sem parâmetros elétricos calibrados). Sob esse modelo a transição se estabiliza em cerca de 1,8 ns, na mesma ordem de grandeza do valor medido no asimut.

## **7. Conclusão**

O multiplicador foi implementado, verificado exhaustivamente (256/256), posicionado, roteado, com atraso de propagação medido. O esforço maior foi no ambiente de ferramentas: binários quebrados, caminhos de arquivo não documentados, um simulador SPICE sem modo não-interativo. Esses problemas ficaram contornados nos scripts do repositório.

Repositório: [github.com/Lucasnobrega38/mult4-alliance-cad](https://github.com/Lucasnobrega38/mult4-alliance-cad)